

● LIVE

หลักสูตรอบรม

CLAUDE 

CODE มือโปร:

ดำน้ำลึกสู่ PRODUCTION



ผู้สอน

แล้วผู้สอนเป็นใคร?



อาจารย์สาวิตร ไกยม

CEO & Founder, IT Genius
Engineering

จบปริญญาโทด้านเทคโนโลยีสารสนเทศ จากมหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ (KMUTNB) อยู่ในวงการพัฒนาโปรแกรมและไอทีมากกว่า 15 ปี

ประสบการณ์: โปรแกรมเมอร์บริษัทเอกชน · ที่ปรึกษาไอทีให้ SME · อาจารย์พิเศษที่ KMUTNB

การสอน: ถ่ายทอดความรู้ด้านโปรแกรมมิ่งและเว็บให้ผู้เรียนหลายพันคน

ความเชี่ยวชาญ: Web Development, DevOps, Next.js, Docker, CI/CD และ Agentic Coding

แนวคิดหลักสูตร

จาก Vibe Coding ที่ทุกคนรู้จัก สู่ Agentic Coding ที่ใช้ได้เต็มศักยภาพ

นักพัฒนาส่วนใหญ่ใช้ Claude Code ได้เพียง 10% ของความสามารถจริง — สิ่งงานที่ละบรรทัด ไม่เคยตั้งค่า Project Memory ไม่รู้จัก Sub-agent ไม่เคยต่อ MCP หลักสูตรนี้พาดำน้ำลึกลงไปถึงแก่นการใช้งานจริงด้วยโปรเจกต์เดียวที่ต่อเนื่องจาก Vibe สู่ Production

Autocomplete → เติมโค้ดทีละบรรทัด



Agentic Coding → อ่าน วางแผน แก้ไข รัน ตรวจสอบ — ด้วยตัวเอง

กำหนดการ

กำหนดการอบรม

5 วันอบรม
เฉลี่ยวันละ 3 ชั่วโมง

15 ชั่วโมงรวม
ตลอดหลักสูตร

20:30–23:30 น. เรียนรอบคำ

วันที่อบรม (2569)

4 – 5 กรกฎาคม

11 – 12 กรกฎาคม

18 กรกฎาคม

สอนสดผ่าน Zoom Meeting พร้อมวิดีโอบันทึกย้อนหลังให้ทบทวนได้ตลอดหลักสูตร

ภาพรวม

ภาพรวมหัวข้อการฝึกอบรม

- 01** แนวคิด Agentic Coding และการใช้ Claude Code อย่างมือโปร
- 02** วาง CLAUDE.md เป็น Project Memory และบริหาร Context Window
- 03** สร้าง Custom Slash Commands และใช้ Plan Mode ออกแบบก่อนลงมือ
- 04** ออกแบบและใช้งาน Sub-agents เฉพาะทางสำหรับ Review, Test, Security
- 05** ต่อ MCP กับเครื่องมือภายนอก และ Automation ด้วย Hooks
- 06** ทำงานเป็นทีมด้วย Claude Code — แชร์ Config ผ่าน Git และ PR Workflow
- 07** Containerize ด้วย Docker วาง CI/CD ด้วย GitHub Actions และ Deploy ขึ้น VPS Ubuntu จริงพร้อม Nginx + HTTPS

ก่อนสมัคร

ผู้เรียนต้องมีพื้นฐานอะไรบ้าง?

ควรมีติดตัวมาบ้าง

- ✓ พื้นฐานเขียนเว็บด้วย JavaScript / TypeScript หรือ React (ไม่ต้องเชี่ยวชาญ)
- ✓ เข้าใจแนวคิดฐานข้อมูลเชิงสัมพันธ์เบื้องต้น
- ✓ เคยใช้ Command Line / Terminal และ Git มาบ้าง
- ✓ มีคอมพิวเตอร์ส่วนตัวและบัญชี Claude ที่ใช้ Claude Code ได้

ไม่จำเป็นต้องมีมาก่อน

Claude Code · Sub-agent · Docker · CI/CD · Linux Server

หลักสูตรปูพื้นฐานให้ครบทุกเรื่องตั้งแต่ศูนย์

จุดเด่น

จุดเด่นของหลักสูตร

เจาะลึกระดับมือโปร

Agentic Loop, Context Management, Sub-agent, MCP และ Hooks
— ใช้ AI ได้เต็มประสิทธิภาพจริง

Sub-agent เป็นหัวใจหลักสูตร

อุทิศทั้งวันให้กับ Sub-agent เฉพาะทาง ทักษะที่หาเรียนฟรีได้ยาก

ออกแบบเพื่อทำงานเป็นทีม

แชร์ CLAUDE.md, Slash Commands, Sub-agent ผ่าน Git ให้ทั้งทีม
มาตรฐานเดียวกัน

เรียนผ่านโปรเจกต์เดียวจบ

พัฒนาระบบคลังสินค้าตั้งแต่บรรทัดแรกจนแอปออนไลน์จริง

ครอบคลุมตั้งแต่ Dev จนถึง Production

ปิดช่องว่างที่บทเรียนทั่วไปมักหยุดแค่ localhost ด้วยการพาไปจนถึง Docker, CI/CD และ VPS จริง

เป้าหมาย

วัตถุประสงค์ของหลักสูตร

- 1 เข้าใจแนวคิด Agentic Coding และใช้ Claude Code พัฒนาเว็บแอปด้วย Next.js อย่างมีประสิทธิภาพ
- 2 ตั้งค่า CLAUDE.md, Custom Slash Commands และบริหาร Context Window ได้อย่างมือโปร
- 3 ออกแบบ Sub-agent เฉพาะทาง ต่อ MCP และตั้งค่า Hooks เพื่อ Automate งานพัฒนา
- 4 ใช้ Claude Code ทำงานเป็นทีมร่วมกับ Git, Code Review และ PR Workflow
- 5 Containerize ด้วย Docker วาง CI/CD และ Deploy ขึ้น VPS พร้อม Nginx ตาม Best Practices

กลุ่มเป้าหมาย

หลักสูตรนี้เหมาะกับใคร?

Web Developer ที่ใช้ AI เขียนโค้ดอยู่แล้ว แต่อยากใช้ Claude Code เพิ่มความสามารถ

Frontend / Full Stack Developer ที่เขียน Next.js เป็น แต่ยังไม่เคย Deploy ขึ้น Server จริง

Tech Lead ที่ต้องการวางมาตรฐานการใช้ AI ให้ทั้งทีม

ผู้ที่อยากเริ่มต้นสาย DevOps ผ่าน Docker และ CI/CD จริง

Freelancer หรือเจ้าของโปรเจกต์ที่ดูแลทั้งพัฒนาและ Deploy เอง

นักพัฒนาที่อยากยกระดับ Productivity ด้วย Sub-agent, MCP และ Agentic Workflow

เครื่องมือ

Tech Stack และเครื่องมือที่ใช้

ฝั่ง AI และการพัฒนา

- Claude Code (Sub-agent · MCP · Hooks)
- Next.js (App Router)
- Prisma ORM
- PostgreSQL

ฝั่ง DevOps และ Production

- Git / GitHub · GitHub Actions
- Docker · Docker Compose
- VPS Ubuntu Linux
- Nginx · Let's Encrypt

BIG PICTURE

ภาพรวมหลักสูตร — เส้นทาง 5 วัน

ทั้ง 5 วันคือโปรเจกต์เดียวที่ต่อเนื่องกัน ไม่ใช่หัวข้อที่แยกจากกัน

Day 1	Day 2	Day 3	Day 4	Day 5
ปูพื้น Claude Code + ตั้ง ต้น StockApp ด้วย Next.js	พัฒนาฟีเจอร์คลั่งสินค้าให้ใช้ งานได้จริง + Slash Commands	ดำน้ำลึก Sub-agents, MCP และ Hooks — หัวใจ ของหลักสูตร	ทำงานเป็นทีม + Docker Container + CI Pipeline	Deploy สู่ Production จริง ด้วย VPS, Nginx และ CD อัตโนมัติ

MODULE 0

เตรียมเครื่องมือก่อนเริ่มเรียน

ติดตั้งเครื่องมือทั้ง 6 อย่างให้พร้อม ก่อนวันเสาร์ที่ 4 กรกฎาคม 2569 เวลา 20:30 น.

0.1

โปรแกรมที่ใช้ในการอบรม

01
Visual Studio Code

02
Node.js 22+

03
pnpm

04
Docker Desktop

05
Git

06
บัญชี Anthropic

เครื่องมือที่ 1 / 6

Visual Studio Code

เวอร์ชันล่าสุด — code.visualstudio.com

<https://code.visualstudio.com>

Extensions ที่แนะนำ

1. Claude Code for VS Code (Anthropic)
2. Markdown Preview Enhanced
3. MySQL by Database Client
4. Catppuccin for VSCode
5. Catppuccin Icons for VSCode
6. Prisma (syntax highlight .prisma)

เครื่องมือที่ 2 / 6

Node.js

20.19+ · แนะนำ 22.x — nodejs.org

การอบรมรองรับตั้งแต่ Node.js 20 ขึ้นไป สามารถใช้ Node.js 21, 22, 23 หรือ 24 ก็ได้ — ดาวน์โหลดตัวติดตั้งจากเว็บไซต์ทางการแล้วรันตามขั้นตอนปกติ

```
$ node --version  
# v20.19.x หรือสูงกว่า
```


pnpm

9.x ขึ้นไป — Package Manager มาตรฐานของหลักสูตร

ตลอดหลักสูตรนี้ใช้ **pnpm** แทน npm หรือ yarn ทุกคำสั่ง — เป็นกฎที่ระบุไว้ใน CLAUDE.md ของโปรเจกต์ด้วย

```
$ npm install -g pnpm  
$ pnpm --version  
# 9.x.x หรือสูงกว่า
```

เครื่องมือที่ 4 / 6

Docker Desktop

เวอร์ชันล่าสุด — docker.com/products/docker-desktop

ใช้สำหรับรับ **PostgreSQL ผ่าน container** ในหลักสูตรนี้ — หากมี PostgreSQL ติดตั้งอยู่แล้วในเครื่องก็ไม่จำเป็นต้องใช้ Docker สำหรับฐานข้อมูล แต่ยังต้องใช้ Docker ในวันที่ 4 สำหรับ Containerize ตัวแอปเอง

Git

เวอร์ชันล่าสุด — git-scm.com

จำเป็นสำหรับการแชร์ **CLAUDE.md**, Slash Commands และ Sub-agent ให้ทั้งทีมใช้ร่วมกัน (วันที่ 4) รวมถึง Git Workflow ร่วมกับ Claude Code

```
$ git --version
# git version 2.x.x
```

บัญชี Anthropic + ตรวจสอบ Environment

สมัครที่ **claude.ai** (Claude Pro/Max) หรือ
console.anthropic.com (API Key) — เลือกวิธีไหนก็ใช้กับ
Claude Code ได้

```
$ node --version
$ pnpm --version
$ git --version
$ docker --version
$ claude --version
```

✓ ตรวจสอบทุกข้อแล้ว → พร้อมเริ่ม Module 1: ทำความรู้จัก Claude Code เชิงลึก

DAY 1 · 4 กรกฎาคม 2569 · 20:30–23:30 น.

ปูพื้น Claude Code ระดับมือโปร และตั้งต้นระบบคลัง สินค้าด้วย Next.js

วิทยากร: อาจารย์สามิตร ไกยม · IT Genius Engineering Co., Ltd.

บทนำ

วันนี้เราจะวางรากฐานที่แน่น ก่อนดำน้ำลึกสู่การพัฒนาจริง

ผู้เรียนจะเข้าใจว่า Claude Code ทำงานอย่างไรในระดับ Agentic Loop ตั้งค่าและใช้งาน Claude Code CLI ได้อย่างมั่นใจ รู้จักเทคนิคเขียน Prompt ที่ดี และลงมือสร้างโปรเจกต์จริง

โปรเจกต์หลักตลอดหลักสูตร

StockApp

ระบบคลังสินค้าเบิกจ่าย พัฒนาด้วย Next.js (App Router), Prisma และ PostgreSQL

ปรัชญาของหลักสูตร

เราไม่ได้พิมพ์โค้ด เราทำกับ AI

ออกแบบ Prompt

บอก Claude ว่าต้องการอะไร อย่างไร จากไฟล์ไหน

ตรวจสอบ (Review)

ดูว่า Claude สร้างสิ่งที่ถูกต้องไหม ตรงกับที่ต้องการไหม

ตัดสินใจ

อนุมัติให้ Claude ดำเนินการต่อ หรือแก้ไข prompt แล้วลองใหม่

โค้ดอ้างอิงในโน้ตวันนี้ไม่ใช่สิ่งที่ต้องพิมพ์ตาม — มีไว้เทียบผลลัพธ์และช่วย review ว่า Claude พลาดจุดไหน

MODULE 1.1

Claude Code คืออะไร?

Claude Code คือ CLI Tool ที่ Anthropic พัฒนาขึ้น ให้นักพัฒนาทำงานร่วมกับ AI โมเดล Claude ได้โดยตรงใน Terminal สิ่งที่ต่างจาก Chatbot ทั่วไปคือมันมีสิทธิ์เข้าถึง **Filesystem** ของคุณ

อ่าน-เขียน-แก้ไขไฟล์ในโปรเจกต์

รับคำสั่ง Terminal ได้เอง

ทำงานหลายขั้นตอนต่อเนื่องอัตโนมัติ

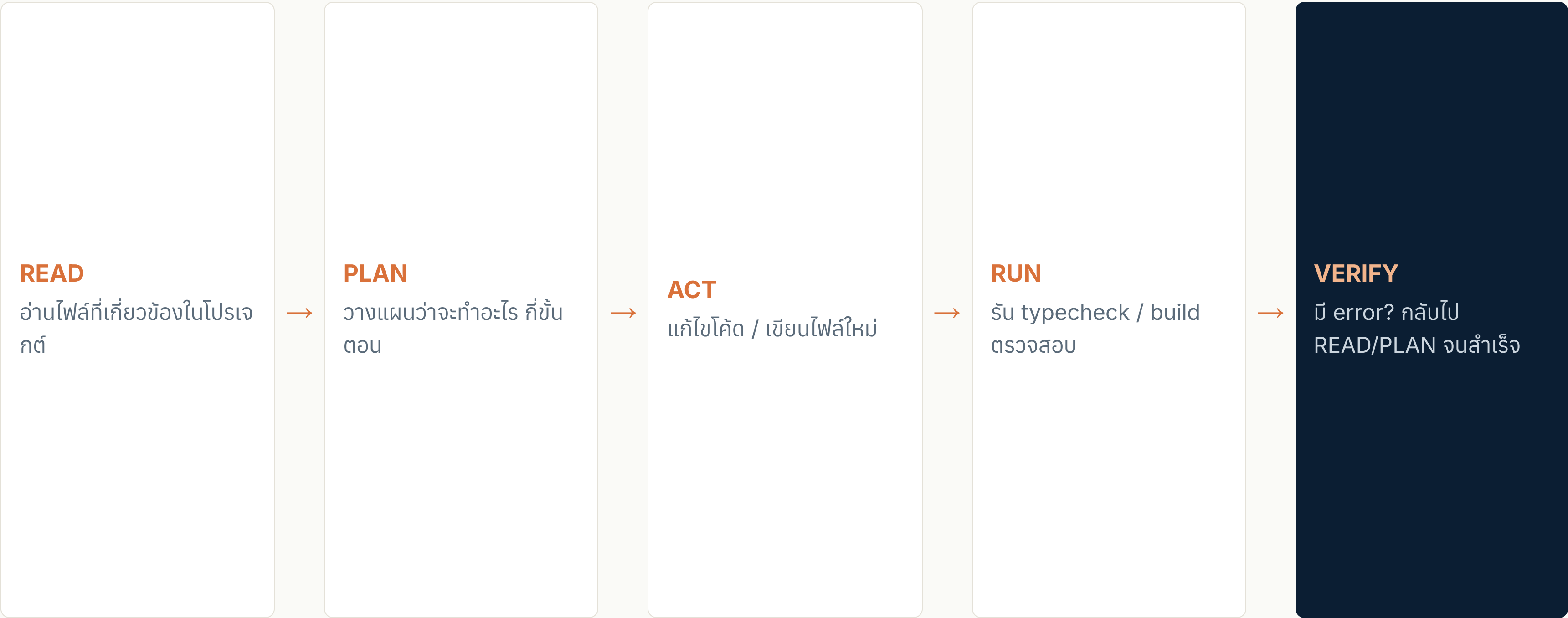
MODULE 1.1

Autocomplete vs Agentic Coding

มิติ	Copilot / Cursor Autocomplete	Claude Code (Agentic)
วิธีการทำงาน	เติมโค้ดตาม cursor ปัจจุบัน	วางแผนและดำเนินงานหลายขั้นตอนเอง
Context	เห็นเฉพาะไฟล์ที่เปิดอยู่	เห็นทั้ง Codebase (grep, find, read)
สิ่งที่ทำได้	เสนอโค้ด — ต้อง Accept เอง	เขียนโค้ด + รับคำสั่ง + แก้หลายไฟล์
Iteration	ต้อง refine เอง	รัน → ดู error → แก้ → รันซ้ำ อัตโนมัติ

MODULE 1.1

Agentic Loop — หัวใจการทำงาน



Claude Code ไม่ได้แค่ "ให้คำตอบ" — มันวนซ้ำ Loop หลายรอบจนกว่างานจะสำเร็จ คล้ายนักพัฒนาจริง

MODULE 1.2

ติดตั้งและ Login Claude Code

```
$ npm install -g @anthropic-ai/claude-code  
$ claude --version  
$ claude
```

เลือกวิธี Authenticate

Claude.ai account — สำหรับ Claude Pro / Max (แนะนำสำหรับหลักสูตรนี้)

Anthropic API key — สำหรับนักพัฒนาที่ควบคุม cost เอง

MODULE 1.2

เลือก Model ให้เหมาะกับงาน

Model	จุดเด่น	เหมาะกับงาน
claude-opus-4	ฉลาดที่สุด, reasoning ลึก	ออกแบบ Architecture, ปัญหาซับซ้อน
claude-sonnet-4	สมดุลความเร็ว/ความแม่นยำ	งานทั่วไป, สร้าง Feature (ใช้บ่อยสุด)
claude-haiku-4	เร็วที่สุด, ประหยัด token	งานง่าย, แก้ typo, คำถามสั้น

MODULE 1.2

Permission Modes — ควบคุมอิสรภาพของ Claude

Mode	คำอธิบาย	เหมาะกับ
default	ถามก่อนทำทุกอย่างที่มีผลกระทบ	โปรเจกต์ใหม่, งานที่ไม่มั่นใจ
acceptEdits	ยอมรับการแก้ไขไฟล์อัตโนมัติ แต่ถามก่อนรับคำสั่ง	งานพัฒนาปกติ (แนะนำ)
plan	วางแผนเท่านั้น ไม่แตะไฟล์จริง	ก่อนเริ่มงานใหญ่ที่ต้องตรวจสอบก่อน
bypassPermissions	ทำทุกอย่างโดยไม่ถาม (อันตราย!)	CI/CD หรือ automation ที่ควบคุมได้ 100%

MODULE 1.3

Vibe Coding อย่างมีวินัย

✗ Prompt ที่ไม่ดี

ทำหน้าแสดงสินค้าให้หน่อย

ปัญหา: ไม่รู้ดึงข้อมูลจากไหน ใช้ component อะไร มี filter ไหน style แบบไหน

✓ Prompt ที่ดี

สร้างหน้า /products (app/products/page.tsx)

- ดึงข้อมูลจาก Prisma (model Product)
- แสดงเป็น table ด้วย Tailwind CSS
- badge สถานะตาม quantity/reorderPoint
- ใช้ Server Component ไม่ใช่ useEffect
- ห้ามใส่ semicolon

หลักการ 5C ของ Prompt ที่ดี

Context บอกบริบทของงานให้ชัด	Constraint ระบุข้อจำกัด/กฎที่ต้องทำตาม	Concrete ระบุ output ที่ต้องการชัดเจน	Context File อ้างอิงไฟล์ให้ Claude อ่านด้วย @	Check ขอให้วางแผน/ยืนยันก่อนลงมือ
--	--	---	---	---

MODULE 1.3

Plan Mode — คิดก่อนทำ

เปิดด้วย **Shift+Tab** ขณะอยู่ใน Claude Code session

ผู้ใช้ [Shift+Tab เข้า Plan Mode]:

สร้าง API Routes: GET /api/products, POST /api/transactions พร้อม error handling

Claude (Plan Mode — ยังไม่แตะไฟล์):

1. อ่าน prisma/schema.prisma, lib/db.ts
2. สร้าง app/api/products/route.ts → prisma.product.findMany()
3. สร้าง app/api/transactions/route.ts → validate + prisma.\$transaction

ยืนยันดำเนินการต่อหรือไม่?

MODULE 1.4

CLAUDE.md — สมอของโปรเจกต์

CLAUDE.md คือไฟล์ที่ Claude Code อ่านโดยอัตโนมัติทุกครั้งที่เราเริ่ม session ใหม่ ทำหน้าที่เหมือนคู่มือทีม ไม่ต้องอธิบาย context ซ้ำทุกครั้ง

- Tech Stack ของโปรเจกต์
- กฎการเขียนโค้ดของทีม
- โครงสร้างโฟลเดอร์ที่ต้อง
- คำสั่ง dev/test/build ที่ใช้บ่อย

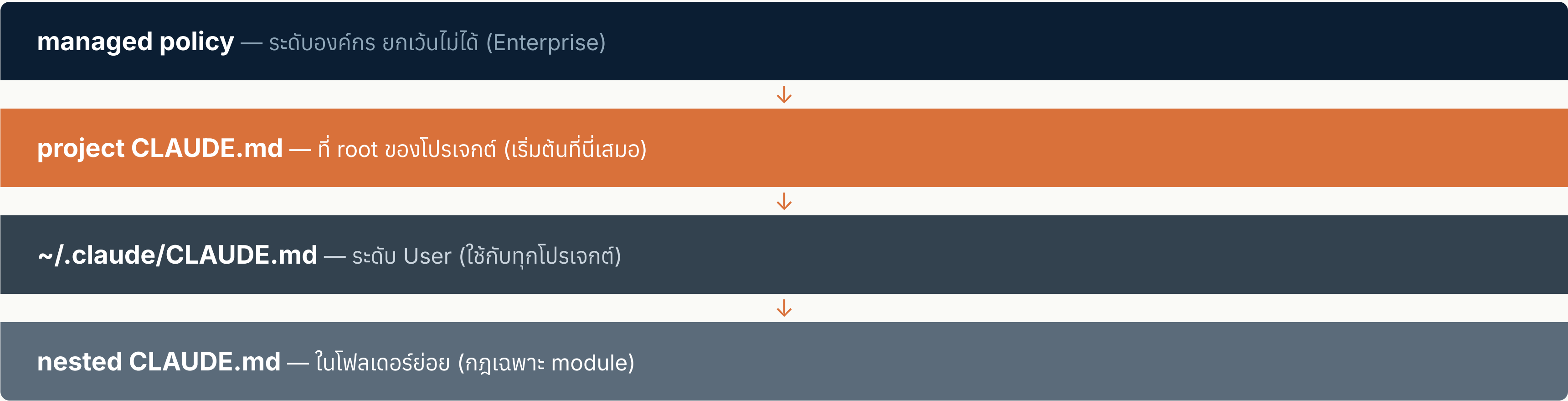
MODULE 1.4

โครงสร้างไฟล์ที่ Claude Code ใช้

```
stock-app/  
├─ CLAUDE.md ← สมอของโปรเจกต์ (อ่านทุกครั้ง)  
├─ prisma.config.ts ← Prisma 7 CLI config  
├─ .claude/ ← โฟลเดอร์ config ของ Claude Code  
│   └─ settings.json ← ตั้งค่าทีม (commit ขึ้น Git ได้)  
│   └─ settings.local.json ← ตั้งค่าส่วนตัว (.gitignore)  
│   └─ skills/ ← Agent Skills (วันที่ 3)  
├─ app/ ← Next.js App Router  
├─ prisma/schema.prisma, migrations/, seed.ts  
└─ src/generated/prisma/ + src/lib/prisma.ts
```

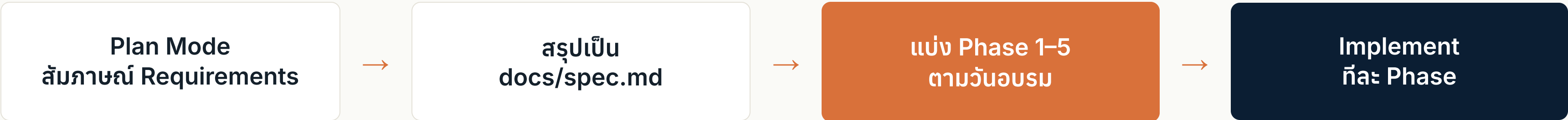

MODULE 1.4

CLAUDE.md — ลำดับชั้น (Hierarchy)



ไม่มีไฟล์ memory.md — CLAUDE.md คือ memory หลักที่เราเขียนและดูแลเอง

Workflow มือโปร: Spec-Driven Development



วิธีนี้ช่วยให้ทุกบรรทัดโค้ดที่ Claude สร้างมีทิศทางชัดเจน ลดการ backtrack และทำให้ทีมทั้งหมดเข้าใจภาพรวมโปรเจกต์ตรงกัน

เข้า Plan Mode สัมภาษณ์ Requirements

[Plan Mode] ทำตัวเป็น PM/Architect
สัมภาษณ์ requirements ของ "StockApp"

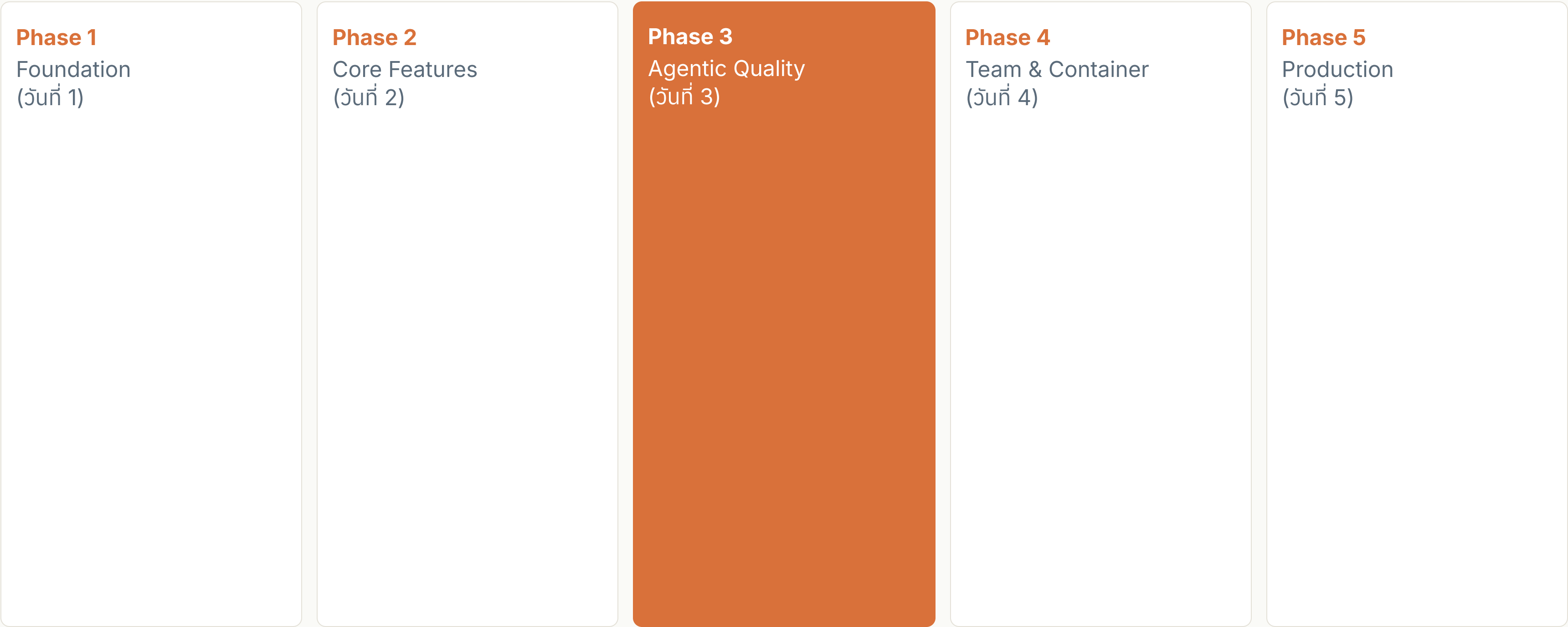
Stack: Next.js 16 + TypeScript + Prisma
+ PostgreSQL + Tailwind v4 + shadcn/ui
ฟีเจอร์: CRUD สินค้า, Stock In/Out,
Dashboard + แจ้งเตือนใกล้เคียง

ถามทีละคำถาม แล้วสรุป requirements

✅ Checkpoint

Claude อยู่ใน Plan Mode ตลอด ไม่แตะไฟล์
Requirements ครอบคลุมครบ 4 ฟีเจอร์หลัก
ยืนยัน requirements ก่อนไปขั้นตอนต่อไป

สร้าง docs/spec.md แบ่ง Phase 1-5



แต่ละ Phase มี Data Model, ไฟล์พร้อม acceptance criteria และ checklist งานย่อยแบบ markdown checkbox

/clear แล้วเริ่ม Implement Phase 1

```
$ /clear
```

ใช้ @docs/spec.md เป็นฐาน

implement Phase 1 ทีละขั้นตาม checklist

หยุด+รายงานให้ review ก่อนไปขั้นต่อไป

ห้ามใส่ semicolon

ทำไมต้อง

/clear

ก่อน? — context การสัมภาษณ์ยาวมากและไม่จำเป็นต่อการ implement
การล้างช่วยลด token ทำให้ตอบเร็วและแม่นยำขึ้น

สร้างโปรเจกต์ Next.js 16

```
pnpm create next-app@latest stock-app \
  --typescript --tailwind --app \
  --src-dir=false \
  --import-alias="@/*"
```

- ✓ Next.js 16 ใช้ Turbopack เป็น default ทั้ง dev และ build
- ✓ ต้องใช้ Node.js 20.9+ (แนะนำ 22.x) + React 19.2
- ✓ Next.js 16 ลบคำสั่ง **next lint** ออกแล้ว — ใช้ ESLint/Biome แทน

ติดตั้ง shadcn/ui (Tailwind CSS v4)

```
pnpm dlx shadcn@latest init
# style: new-york, base: neutral
pnpm dlx shadcn@latest add \
  button card input label badge table select
```

- ✓ postcss.config.mjs ใช้ '@tailwindcss/postcss'
- ✓ globals.css มี @import "tailwindcss" + CSS variables
- ✓ src/lib/utils.ts มีฟังก์ชัน cn (clsx + tailwind-merge)
- ✓ src/components/ui/ มี component ครบ 7 ตัว

ติดตั้ง Prisma 7 + PostgreSQL Deps

```
pnpm add -D prisma @types/pg tsx
pnpm add @prisma/client @prisma/adapter-pg pg dotenv
pnpm prisma init --datasource-provider postgresql \
  --output ../src/generated/prisma
```

- ✓ สร้าง prisma/schema.prisma, .env
- ✓ สร้าง prisma.config.ts (ไฟล์ใหม่ใน Prisma 7)
- ✓ generator ใช้ provider "prisma-client" ไม่ใช่แบบเดิม

ตั้งค่า PostgreSQL ด้วย Docker

```
docker run --name stockapp-postgres \  
-e POSTGRES_USER=stockuser \  
-e POSTGRES_PASSWORD=stockpass \  
-e POSTGRES_DB=stockdb \  
-p 5432:5432 -d postgres:16
```

```
# .env  
DATABASE_URL=  
"postgresql://stockuser:stockpass  
@localhost:5432/stockdb?schema=public"
```

เขียน Prisma Schema

```
enum TransactionType { IN OUT }

model Product {
  id Int @id @default(autoincrement())
  sku String @unique
  name String category String unit String
  quantity Int @default(0) reorderPoint Int @default(10) price Int @default(0)
  transactions StockTransaction[]
  @@map("products")
}

model StockTransaction {
  id Int @id @default(autoincrement()) productId Int type TransactionType
  quantity Int note String?
  product Product @relation(fields: [productId], references: [id], onDelete: Cascade)
  @@map("stock_transactions")
}
```

Migration + Generate Client

```
pnpm prisma migrate dev --name init
pnpm prisma generate
```



Prisma 7:

migrate dev

ไม่ generate client และ
รับ seed อัตโนมัติแล้ว —
ต้องรับ

prisma generate

prisma db seed

แยก
เอง

Prisma Client Singleton

```
// src/lib/prisma.ts
import { PrismaPg } from '@prisma/adapter-pg'
import { PrismaClient } from '@generated/prisma/client'

const sc-camel-global-for-prisma = globalThis as unknown as { prisma?: PrismaClient }
const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL })

export const prisma = globalForPrisma.prisma ?? new PrismaClient({ adapter })
if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma
```

globalThis ป้องกัน connection pool leak จาก Hot Reload ของ Next.js Dev Server

สร้าง Seed Data

7

สินค้าตัวอย่าง

9

Transaction ตัวอย่าง

SKU-1001 ถึง SKU-1007 ครอบคลุมสถานะสต็อกครบ 3 แบบ (ปกติ / ใกล้เคียง / หมดสต็อก) — ลบข้อมูลเดิมด้วย StockTransaction ก่อน Product เพราะมี relation

รัน Seed + ตรวจสอบด้วย Prisma Studio

```
pnpm db:seed  
pnpm prisma studio  
# เปิด http://localhost:5555
```

- ✓ ตาราง products มี 7 แถว
- ✓ ตาราง stock_transactions มี 9 แถว
- ✓ ปิด Prisma Studio ด้วย Ctrl+C หลังตรวจสอบเสร็จ

สร้าง CLAUDE.md ด้วย /init

```
$ claude  
> /init
```

- ✓ Tech Stack คสย: Next.js 16, Prisma 7, Tailwind v4, pnpm
- ✓ กฎ: ห้ามใส่ semicolon ทุกไฟล์
- ✓ Naming convention: Component=PascalCase, utility=camelCase
- ✓ Logic สถานะสืบทอด + Prisma model names ถูกต้อง
- ✓ ข้อห้าม: ห้ามแก้ migrations, ห้าม commit .env

วางโครงสร้าง .claude/settings.json

```
{  
  "model": "claude-sonnet-4",  
  "permissions": {  
    "allow": ["Read(*)", "Write(*)", "Bash(pnpm *)", "Bash(git status)"],  
    "ask": ["Bash(pnpm prisma migrate *)", "Bash(docker *)", "Bash(rm *)"]  
  },  
  "hooks": {}  
}
```

settings.json commit ขึ้น Git ได้ — settings.local.json เป็นของส่วนตัว ต้องอยู่ใน .gitignore

ทดสอบว่า CLAUDE.md ทำงานจริง

```
// src/lib/utils.ts — ต่อย้าย cn() ที่ shadcn สร้างไว้
export function getStockStatus(quantity, reorderPoint) {
  if (quantity === 0) return 'หมดสต็อก'
  if (quantity <= reorderPoint) return 'ใกล้หมด'
  return 'ปกติ'
}
export function formatPrice(price) { ... }
export function formatDate(date) { ... }
```

Claude อ้างอิง logic จาก CLAUDE.md ได้ถูกต้องทันที — นี่คือหลักฐานว่า CLAUDE.md ทำงานเป็น "สมอง" ของโปรเจกต์จริง

สำรวจ StockApp ด้วยตัวเอง

ภารกิจที่ 1 — API Route แรก

สร้าง GET /api/products — ดึงสินค้าทั้งหมดผ่าน Prisma, return { data, total }, จัดการ error ด้วย try/catch

ภารกิจที่ 2 — Plan Mode: Dashboard

กด Shift+Tab แล้ววางแผน Dashboard แสดง summary cards + ตารางสินค้าล่าสุด 5 รายการ

สรุป

โครงสร้างโปรเจกต์สิ้นสุดวันที่ 1

```
stock-app/  
├─ CLAUDE.md · prisma.config.ts · components.json  
├─ docs/spec.md ← Phase 1-5  
├─ .claude/settings.json ← permissions + model  
├─ .env · .env.example · .gitignore  
├─ app/api/products/route.ts · layout.tsx · page.tsx · globals.css  
├─ src/components/ui/ (7 shadcn components)  
├─ src/generated/prisma/ ← gitignored  
├─ src/lib/prisma.ts · utils.ts  
└─ prisma/schema.prisma · migrations/ · seed.ts
```


สรุปวันที่ 1

วันนี้เราได้เรียนรู้และลงมือทำ

✓ เตรียม Environment ควบ + Login Claude Code สำเร็จ

✓ เลือก Model และ Permission Mode ได้เหมาะสม

✓ สร้าง docs/spec.md แบ่ง Phase 1–5 (Spec-Driven)

✓ วางโครง .claude/ (settings.json) ตั้งแต่นั้นโปรเจกต์

✓ เข้าใจปรัชญา Vibe Coding และ Agentic Loop

✓ เขียน Prompt ที่ดีด้วยหลักการ 5C + ใช้ Plan Mode

✓ สร้างและปรับปรุง CLAUDE.md — สมอของโปรเจกต์

✓ Implement Phase 1 ควบ: StockApp + Prisma + Seed 7 สีนค้

พรุ่งนี้ (วันที่ 2): Phase 2 — Core Features · CRUD, Stock In/Out, Dashboard และ Custom Slash Commands

ขอบคุณครับ

ไว้เจอกันต่อในวันที่ 2



อาจารย์สาวิตร ไกยม
IT Genius Engineering

DAY 2 · 5 กรกฎาคม 2569 · 20:30–23:30 น.

พัฒนาพีเจอร์คำสั่งให้ใช้งานจริง พร้อมเทคนิค Context และ Slash Commands

วิทยากร: อาจารย์สามิตร โทยม · IT Genius Engineering Co., Ltd.

จากรากฐาน สู่ฟีเจอร์ที่ใช้ได้จริงในองค์กร

วันที่ 1 เราสร้าง StockApp (Next.js 16 + TypeScript + Prisma + PostgreSQL) เรียบร้อยแล้ว วันนี้เริ่มด้วยระบบ **Authentication** ก่อน จากนั้นสร้างฟีเจอร์จริง — CRUD สินค้า, รับเข้า-เบิกจ่ายสต็อก, และ Dashboard พร้อมเรียนเทคนิคขั้นสูง 2 อย่าง



Better Auth

login / register / guard ทุกหน้า



Core Features

CRUD · Stock In/Out · Dashboard



เทคนิคขั้นสูง

Context Window · Custom Slash Commands

เปิดวัน · โหลด SPEC

ล๊อกขอบเขต Phase 2 ก่อนลงมือ

```
$ /clear
```

อ่าน @docs/spec.md แล้วสรุปสั้น ๆ
ว่า Phase 2 (Core Features) มีงานย่อยอะไร
ยืนยันว่า Phase 1 เสร็จครบแล้ว
ขอให้ฉันสั่งทำที่ละงาน อย่าเพิ่งลงมือ

งานย่อยใน Phase 2

- ระบบ Auth (Better Auth) + guard ทุกหน้า
- CRUD จัดการสินค้า
- Stock In / Out กันเบิกเกิน
- Dashboard + แจ้งเตือนใกล้หมด
- Custom Slash Commands

Key: เปิด spec ต้นวันแล้ว /clear ทำให้ context สะอาด โฟกัสเฉพาะ Phase ที่ทำ

พร้อมจากวันที่ 1

โครงสร้างที่ต้องมีอยู่แล้วก่อนเริ่ม

```
stock-app/
├─ prisma/schema.prisma ← Product/StockTransaction
├─ prisma/seed.ts ← SKU-1001..1007
├─ prisma.config.ts ← Prisma 7
├─ src/lib/prisma.ts ← singleton + adapter
├─ src/generated/prisma/ ← generate
├─ .env ← DATABASE_URL
└─ .claude/CLAUDE.md
```

```
cd stock-app
pnpm dev
```

Prisma 7: client สร้างด้วย driver adapter — แต่โค้ดฝั่งพีเจอรียังเรียกผ่าน `import { prisma } from '@lib/prisma'` เหมือนเดิมทุกที่ adapter ถูกซ่อนใน singleton ที่เดียว

MODULE 2.0

ระบบ Authentication ด้วย Better Auth

ทำไมต้อง Auth ก่อน?

ทุกฟีเจอร์ (CRUD, Stock, Dashboard) เข้าถึงได้เฉพาะผู้ที่ login แล้ว
— ทำก่อนไม่ต้องย้อนมาเพิ่ม guard ภายหลัง

Better Auth คืออะไร?

Auth library สำหรับ TypeScript รองรับ email/password, session และเชื่อม Prisma ผ่าน adapter — ไม่ต้องเขียน auth logic เอง

Module 2.0 จะสร้าง:

```
├─ lib/auth.ts ← config
├─ lib/auth-client.ts ← hooks
├─ api/auth/[...all]/ ← handler
├─ proxy.ts ← guard ทุกหน้า
└─ (auth)/ login·register·forgot·reset
UI ใช้ shadcn/ui
```

MODULE 2.0 · ขั้นที่ 1

ติดตั้งและตั้งค่า Better Auth

ติดตั้ง better-auth และ:

1. เพิ่ม env: BETTER_AUTH_SECRET (≥ 32),
BETTER_AUTH_URL, NEXT_PUBLIC_APP_URL
2. สร้าง lib/auth.ts email/password
+ Prisma adapter
3. ห้ามมี semicolon

✅ ใช้ prisma ที่ import จาก @/lib/prisma — **ไม่ใช่** new PrismaClient() เปล่า

```
// src/lib/auth.ts
export const auth = betterAuth({
  database: prismaAdapter(prisma, {
    provider: 'postgresql'
  }),
  emailAndPassword: {
    enabled: true,
    sendResetPassword: async ({ user, url }) => {
      console.log('[DEV] Reset: ' + url)
    }
  }
})
```


เพิ่ม Prisma Models ของ Auth + Migrate

user email (unique), role, image?...	session token (unique), expiresAt, userId
account providerId, userId, password?	verification identifier, value, expiresAt

ทุก model ใช้ @@map เป็น snake_case

```
$ pnpm prisma migrate dev \
  --name add-auth-models

$ pnpm prisma generate

# สร้างตาราง user, session,

# account, verification
```

MODULE 2.0 · ขั้นที่ 3

สร้าง API Route และ Auth Client

```
// api/auth/[...all]/route.ts
import { toNextJsHandler } from
  'better-auth/next-js'
import { auth } from '@lib/auth'

export const { POST, GET } =
  toNextJsHandler(auth)
```

```
// src/lib/auth-client.ts
'use client'
export const sc-camel-auth-client = createAuthClient({
  baseURL: process.env.NEXT_PUBLIC_APP_URL
})

export const { signIn, signUp,
  signOut, useSession } = authClient
```

หมายเหตุ: reset รหัสผ่านใช้ `authClient.requestPasswordReset({ email, redirectTo })` และ `resetPassword({ newPassword, token })`

MODULE 2.0 · ชั้นที่ 4

ป้องกันทุกหน้าด้วย Proxy (Next.js 16)

Next.js 16 เปลี่ยน **middleware** → **proxy** · ไฟล์ `src/proxy.ts` · รันบน Node.js runtime

```
// src/proxy.ts
export function proxy(request: NextRequest) {
  const { pathname } = request.nextUrl
  const sc-camel-is-public = publicPaths.some((p) =>
    pathname.startsWith(p))
  const cookie = getSessionCookie(request)
  if (!cookie && !isPublic) return redirect('/login')
  if (cookie && isPublic) return redirect('/')
  return NextResponse.next()
}
```

✅ Checkpoint

ไฟล์ชื่อ `proxy.ts` · function ชื่อ `proxy`

เปิดหน้าใดโดยไม่ login → redirect `/login`

login แล้วเข้า `/login` → redirect `/`

public: `/login /register /forgot /reset`

สร้างหน้า Auth ด้วย shadcn/ui

login

email + password → `signIn.email()` → redirect /

register

name + email + password → `signUp.email()` → /login

forgot-password

email → `requestPasswordReset()` · ดู link ใน console

reset-password

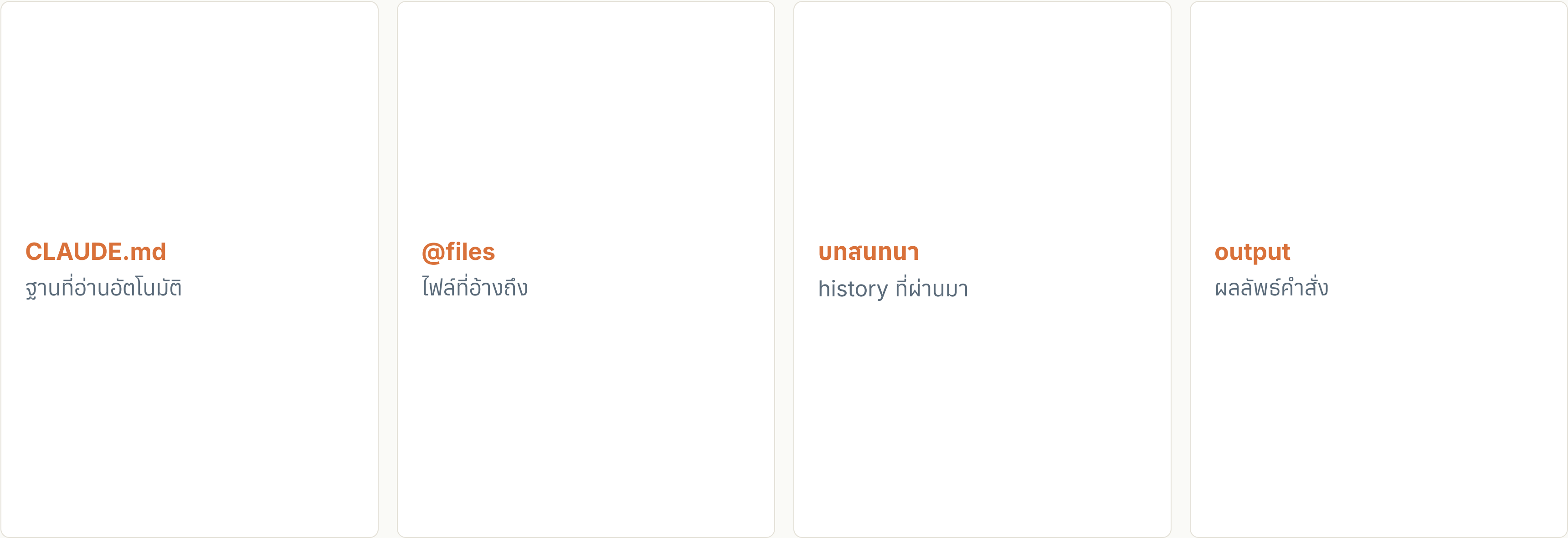
token จาก query → `resetPassword({ newPassword, token })`

ทุกหน้าใช้ Card · Input · Label · Button ของ shadcn/ui + แสดง loading state และ error message ชัดเจน · ห้ามมี semicolon

MODULE 2.1

Context Window คืออะไร?

สิ่งที่ Claude "มองเห็น" ในการตอบแต่ละครั้ง — กล่องหน่วยความจำชั่วคราวที่มีขนาดจำกัด (วัดเป็น tokens)



MODULE 2.1

ปัญหาเมื่อ Context เต็ม

- Claude "ลืม" ว่า schema มีโครงสร้างอะไร
- เขียนโค้ดซ้ำหรือขัดแย้งกับที่มีอยู่
- response ช้าลงเรื่อย ๆ
- token ล้น → error หรือถูก truncate ตอนต้น

Key Concept

Context Window **ไม่ใช่หน่วยความจำถาวร** — เมื่อเต็มแล้ว Claude จะลืมสิ่งที่คุยไปก่อนหน้านี้ และอาจเขียนโค้ดผิดพลาดโดยไม่รู้ตัว

MODULE 2.1

เครื่องมือบริหาร Context

/clear

ล้าง Context ทั้งหมด

เริ่ม feature ใหม่ · context ใกล้เคียง · Claude เริ่ม
สับสน

⚠ ต้อง brief ใหม่ผ่าน CLAUDE.md / @files

/compact

บีบอัดอัจฉริยะ

สรุปบทสนทนาให้กระชับ แต่ยังเก็บสิ่งสำคัญ เช่น
decisions และโครงสร้างโค้ดที่ตกลงกัน

@file

Reference ไฟล์ตรง ๆ

แทน copy-paste — Claude อ่านไฟล์เข้า
context อัตโนมัติ

@prisma/schema.prisma
@lib/prisma.ts

Reference เฉพาะสิ่งที่เกี่ยวข้อง

✅ สิ่งที่ต้องทำ

@schema.prisma ตอนถามเรื่อง DB

/compact ก่อน context เต็ม

Brief ใน CLAUDE.md ครึ่งเดียว

/clear เมื่อเปลี่ยน feature ใหม่

ถามทีละงาน ชัดเจน

⚠ สิ่งที่ต้องเลี่ยง

@ ทุกไฟล์ใน project

รอให้ context เต็มแล้วค่อยทำ

อธิบายซ้ำในทุก prompt

ต่อ session เดียวตลอดวัน

ถามหลายงานพร้อมกัน

อัปเดต CLAUDE.md ให้ครอบคลุม StockApp

อัปเดต @.claude/CLAUDE.md ให้ครบ:

- tech stack (Next 16, Prisma, pnpm...)
- กฎ: ห้าม semicolon, Server Actions, Zod
- Prisma models: Product, StockTransaction
- ตรรกะสถานะสต็อก 3 กรณี
- โครงสร้าง folder หลัก

StockApp — คลังสินค้าเบิกจ่าย

กฎสำคัญ

- ห้ามใส่ semicolon (;)
- ใช้ Server Actions สำหรับ mutation
- ใช้ Zod validate input ก่อนทุกครั้ง

ตรรกะสถานะสต็อก

- quantity == 0 → "หมดสต็อก"
- quantity <= reorderPoint → "ใกล้หมด"
- อื่นๆ → "ปกติ"

MODULE 2.2

Custom Slash Commands

สร้างคำสั่งของตัวเองด้วยไฟล์ `.md` ใน `.claude/commands/` แล้วเรียกใช้ใน terminal ได้ทันที

```
/create-crud Product  
/add-feature stock-alert  
/review-code app/products/actions.ts
```

frontmatter (---) ระบุ description · **\$ARGUMENTS** รับค่าที่พิมพ์ตามหลัง command

```
stock-app/  
└─ .claude/  
    └─ CLAUDE.md  
        └─ commands/  
            ├── create-crud.md  
            ├── add-feature.md  
            └─ review-code.md
```

MODULE 2.2

ทำไม Custom Commands ถึงสำคัญ?

ปัญหาเดิม

พิมพ์ prompt ยาวซ้ำ ๆ ทุกครั้ง
แต่ละคนในทีมมี prompt ต่างกัน
ลืม convention ของโปรเจกต์
Onboard คนใหม่ยาก

แก้ด้วย Custom Commands

พิมพ์ /create-crud Product ครั้งเดียว
ทุกคนใช้ command เดียวกัน ผลสม่ำเสมอ
ฝัง convention ไว้ใน command แล้ว
คนใหม่ใช้ command ได้เลย

Key Concept: Custom Slash Commands คือ "สูตรสำเร็จ" ของทีม — เขียนครั้งเดียว ทุกคนใช้ได้เหมือนกัน + commit เข้า Git เป็นสินทรัพย์ของทีม

MODULE 2.2 · ขั้นตอน

สร้าง Command `/create-crud`

```
// .claude/commands/create-crud.md
---
description: สร้าง CRUD ครบชุด
  สำหรับ Prisma model ที่ระบุ
---
```

สร้าง CRUD สำหรับ model "\$ARGUMENTS"

1. อ่าน @prisma/schema.prisma ก่อน
2. สร้าง page.tsx / new / [id]/edit
3. actions.ts: getAll, get, create, update, delete + Zod

✅ Checkpoint

โฟลเดอร์ .claude/commands/ ถูกสร้าง

ไฟล์มี --- frontmatter + description:

มี \$ARGUMENTS ปรากฏในเนื้อหา

กฎ: ห้าม semicolon, 'use server', revalidatePath, return { success, error }

Command `/add-feature` และ `/review-code`

`/add-feature`

รับ \$ARGUMENTS เป็นชื่อฟีเจอร์
วิเคราะห์ @schema.prisma + @CLAUDE.md ว่ากระทบ model ไหน
แสดง plan เป็น checklist → **รอยืนยันก่อนลงมือ**

`/review-code`

รับ path ไฟล์ผ่าน @\$ARGUMENTS
ตรวจ Correctness · Convention (no semicolon) · Error Handling ·
Performance · Security
แสดงผล  ดีแล้ว /  ควรปรับ /  ต้องแก้ด่วน

MODULE 2.3 · ขั้นตอนที่ 4

CRUD สินค้า — เริ่มด้วย Zod Schema

Next.js 16 แนะนำ **Server Actions** สำหรับ mutation แทน API Routes —
เขียนน้อยกว่า, type-safe, integrate กับ form ตรง ๆ

✅ ใช้ `z.coerce.number()` รับค่าจาก FormData (string) ได้ ·
`.regex(/^SKU-\d+$/)` ตรวจสอบ format

```
// lib/validations/product.ts
export const sc-camel-product-schema = z.object({
  sku: z.string().min(1)
    .regex(/^SKU-\d+$/, 'SKU-XXXX'),
  name: z.string().min(1).max(100),
  category: z.string().min(1),
  quantity: z.coerce.number().int().min(0),
  reorderPoint: z.coerce.number().int().min(0),
  price: z.coerce.number().int().min(0),
})
export type ProductInput = z.infer<typeof productSchema>
```

MODULE 2.3 · ขั้นตอนที่ 5

Server Actions สำหรับ Product

```
// app/products/actions.ts
'use server'

export async function createProduct(formData) {
  const parsed = productSchema.safeParse(raw)
  if (!parsed.success) return { success: false,
    error: ... }
  const existing = await prisma.product
    .findUnique({ where: { sku } })
  if (existing) return { success: false,
    error: 'SKU มีในระบบแล้ว' }
  const product = await prisma.product.create(...)
  revalidatePath('/products')
  return { success: true, data: product }
}
```

✓ Checkpoint

บรรทัดแรกคือ 'use server'

createProduct เช็ค SKU ซ้ำก่อน create

ทุก action มี try/catch

revalidatePath หลัง mutation ทุกครั้ง

getAll · get · create · update · delete

MODULE 2.3 · ขั้นตอนที่ 6

หน้า List สินค้าและฟอร์ม

- page.tsx**

async Server Component · ตาราง + badge สถานะ
- new/page.tsx**

'use client' · useState loading/error
- DeleteProductButton.tsx**

confirm() ก่อนเรียก deleteProduct

```
function getStockStatus(qty, reorder) {
  if (qty === 0) return
    { label: 'หมดสต็อก', color: 'red' }
  if (qty <= reorder) return
    { label: 'ใกล้หมด', color: 'yellow' }
  return { label: 'ปกติ', color: 'green' }
}
```


MODULE 2.4

Business Logic: รับเข้า & เบิกจ่าย

↑ Stock IN (รับเข้า)

`quantity += tx.quantity`

ทำได้เสมอ — ไม่มีข้อจำกัด

↓ Stock OUT (เบิกออก)

ต้องตรวจสอบก่อน!

ถ้า `quantity ≥ ที่ขอ` → ตัดสต็อก 

ถ้า `quantity < ที่ขอ` → throw Error

"สต็อกไม่เพียงพอ" 

Key: ทั้งสองทำใน `prisma.$transaction` เดียว (atomic) — บันทึก transaction + อัปเดต quantity สำเร็จหรือล้มเหลวพร้อมกัน ไม่มีทางค้างกลางคัน

MODULE 2.4 · ขั้นตอนที่ 7

stockOut — กันเบิกเกินด้วย \$transaction

```
await prisma.$transaction(async (tx) => {  
  const product = await tx.product  
    .findUnique({ where: { id: productId } })  
  if (!product) throw new Error('ไม่พบสินค้า')  
  if (product.quantity < quantity)  
    throw new Error('สต็อกไม่เพียงพอ -  
คงเหลือ ' + product.quantity + '...')  
  await tx.stockTransaction.create({  
    data: { productId, type: 'OUT', quantity } })  
  await tx.product.update({ where: { id },  
    data: { quantity: { decrement: quantity } } })  
})
```

✅ Checkpoint

stockIn ใช้ { increment: quantity }

stockOut findUnique ภายใน \$transaction ก่อนเช็ค

Error บอกจำนวนที่มีและที่ขอชัดเจน

catch จับด้วย error instanceof Error

revalidatePath 3 เส้น: /transactions /products /dashboard

หน้าฟอร์ม Stock In / Out + ทดสอบ

ฟอร์มเดียว 2 โหมด

- toggle: รับเข้า (เขียว) / เบิกออก (ส้ม)
- อ่าน ?type=in/out เป็นค่า default
- useEffect โหลด products ตอน mount
- เลือก product → แสดง "คงเหลือ: X หน่วย"
- สำเร็จ → success + reset form

ทดสอบกับเบิกเกิน

กระดาษ A4 quantity = 5

เบิก 3 → $5 \geq 3$  เหลือ 2

เบิก 10 → $2 < 10$ 

"สต็อกไม่เพียงพอ – คงเหลือ 2

แต่ขอเบิก 10"

→ ไม่มี record · quantity คงเดิม

หน้า Dashboard และรายงานสต็อก

สินค้าทั้งหมด

7

สต็อกปกติ

4

ใกล้หมด / หมดแล้ว

3

⚠️ สินค้าที่ต้องดูแล

SKU-1002 กระดาษ A4 · เหลือ 5/10 · ใกล้หมด

SKU-1005 น้ำดื่ม · เหลือ 0/20 · หมดสต็อก

ประวัติล่าสุด

↑ IN กระดาษ A4 +50 · 10 นาทีที่แล้ว

↓ OUT โป๊ตบุ๊ค -2 · 1 ชม.ที่แล้ว

MODULE 2.5 · ขั้นตอนที่ 9

สร้างหน้า Dashboard

```
const [products, recentTransactions] =
  await Promise.all([
    prisma.product.findMany({
      orderBy: { quantity: 'asc' } },
    prisma.stockTransaction.findMany({ take: 10,
      orderBy: { createdAt: 'desc' }, include: {...} }},
  ])
const sc-camel-normal-count = products.filter((p) =>
  p.quantity > p.reorderPoint).length
const sc-camel-low-stock-products = products.filter((p) =>
  p.quantity <= p.reorderPoint)
```

✅ Checkpoint

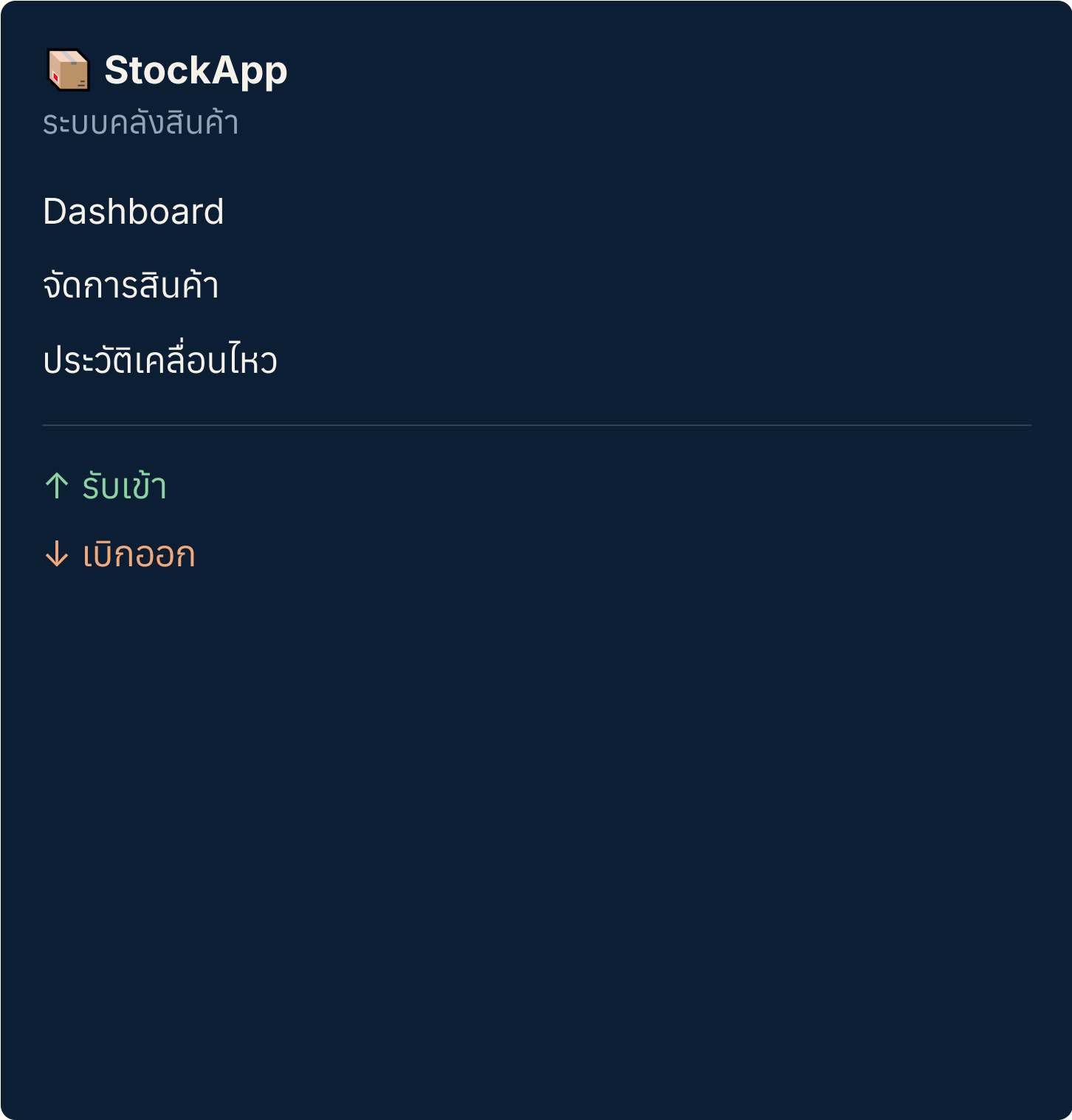
Promise.all ดึงข้อมูลพร้อมกัน

lowStock ใช้ $\leq$ reorderPoint (ไม่ใช่ $<$)

timeAgo คسบ: วินาที นาที ชั่วโมง วัน

card ใกล้เคียงเปลี่ยนสีเมื่อ alertCount > 0

เพิ่ม Navigation Layout



 Checkpoint

- Layout เป็น flex ที่ระดับ body wrapper
- Sidebar กว้าง w-56 และ min-h-screen
- Main content flex-1 overflow-auto
- ลิงก์ /transactions/new?type=in และ ?type=out ถูกต้อง

Debug กับ Claude Code อย่างเป็นระบบ

Prompt Debug ที่ดี — คสว 3 ส่วน

1. Error message เต็ม ๆ (พร้อม stack)
2. สถานการณ์ที่เกิด (reproduce steps)
3. ไฟล์ที่เกี่ยวข้องด้วย @

อย่าถามแค่ "ทำไม error?"

```
# ตัวอย่างฝึก debug
ทดสอบเบิก 100 ขึ้น จากที่มีแค่ 5
แต่ไม่มี error message แสดง
กลับ submit สำเร็จ

ดู @app/transactions/actions.ts
และ @app/transactions/new/page.tsx
หาว่า error ไปไหน แก้ให้แสดงถูกต้อง
```

Refactor ให้ DRY + อธิบายก่อนแก้

Refactor DRY

ปุ่ม Stock In/Out มี logic เหมือนกัน (ต่างแค่สี) → extract เป็น config object หรือ component แยก

ยังคงพฤติกรรมเดิมทุกอย่าง · ห้าม semicolon

อธิบายก่อนแก้

ให้ Claude อธิบาย \$transaction ทีละขั้นก่อนแก้ ช่วยให้:

- 1. ตรวจสอบว่า Claude เข้าใจถูกก่อนแก้
- 2. ค้นพบ misunderstanding ก่อน implement ผิด
- 3. ได้เรียนรู้ไปด้วย

WORKSHOP ทำวันนี้

ต่อยอด StockApp ให้ครบฟีเจอร์

2.1 ทดสอบ Custom Commands

/create-crud Product → เทียบกับ actions.ts ที่สร้างไว้

2.2 Stock In/Out 3 Scenario

รับเข้า → เบิกปกติ → เบิกเกิน (ต้องเห็น error, quantity ไม่เปลี่ยน)

2.3 ฝึก Debug จริง

ลบ guard check ออก → เกิดอะไร? · remove revalidatePath → Dashboard เป็นยังไง?

2.4 หน้าประวัติ Transaction

async Server Component · badge IN เขียว / OUT ส้ม

สรุปวันที่ 2

วันนี้เราได้เรียนรู้และลงมือทำ

✓ วางระบบ Auth (Better Auth) + guard ทุกหน้าด้วย proxy

✓ เขียน CLAUDE.md ที่ดี — ให้ Claude รู้จักโปรเจกต์

✓ พัฒนา CRUD สิ้นค้า — Server Actions + Zod + Prisma

✓ สร้าง Dashboard — สถิติ, แจ้งเตือน, ประวัติ

✓ เข้าใจ Context Window และใช้ /clear, /compact, @file

✓ สร้าง Custom Slash Commands + \$ARGUMENTS

✓ Stock In/Out — กันเบิกเกินด้วย prisma.\$transaction

✓ ฝึก Debug และ Refactor อย่างเป็นระบบ

พรุ่งนี้ (วันที่ 3): Phase 3 — Sub-agents, MCP (Model Context Protocol) และ Hooks · ทำงานอัตโนมัติแบบ multi-step, เชื่อม external tools, ตั้ง automation เบื้องหลัง

Claude Code Cheat Sheet

Keyboard Shortcuts

!	Bash mode prefix
@	Mention files/folders
Esc	Interrupt Claude
Esc · Esc	History navigation
Shift+Tab	Auto-accept ("yolo mode")
Shift+Tab+Tab	Plan mode
Ctrl+V	Paste image

🚀 คำสั่งติดตั้งที่จำเป็น

```
npm install -g @anthropic-ai/claude-code
claude update
claude config set model claude-opus-4-1-20250805
claude config set -g theme dark
```

⚡ Slash Commands ที่ใช้บ่อย

/clear — ล้างประวัติ	/init — เริ่มโปรเจกต์
/model — เปลี่ยน AI model	/config — ดู/แก้ไข config
/memory — แก้ CLAUDE.md	/agents — จัดการ Sub-agent
/mcp — MCP servers	/permissions — ความปลอดภัย

ไฟล์ · Git Worktrees · MCP · Hooks · Models

 ตำแหน่งไฟล์สำคัญ

~/.claude/settings.json
User global settings
.claude/settings.json
Project settings
.claude/commands/
Custom slash commands
.claude/agents/
Project subagents

 Configuration

```
claude config list
claude config set <key> <value>
claude config add <key> <value>
```

 Git Worktrees

```
git worktree add ../app-feature -b
feature main
git worktree list
git worktree remove <path>
```

 MCP Servers ยอดนิยม

```
claude mcp add playwright npx
@playwright/mcp@latest

claude mcp add --transport http
context7
https://mcp.context7.com/mcp
```

 Hook Events

PreToolUse · PostToolUse
UserPromptSubmit · Stop
SessionStart · SessionEnd

ขอบคุณครับ

ไว้เจอกันต่อในวันที่ 3



อาจารย์สาวิตร ไกยม
IT Genius Engineering